

The Mathematics of Backpropagation: Companion Derivations for `llm.moj`

Evan Owen

evan@jwo3.io

Georgia Institute of Technology

Abstract—Working derivations of the backward pass for every operation in the GPT-2 forward pass, as implemented by the `llm.moj` kernels. Sections are ordered by backward-derivation pedagogy—starting from the scalar loss and working toward the inputs—rather than by `llm.c` forward order, so that each gradient is derived only after the one it depends on. Each section anchors on the forward computation and leaves the derivation as structured workspace.

CONTENTS

I	Notation and Conventions	1
II	Warm-up: Scalar Chain Rule on a Tiny Network	2
III	Cross-Entropy Loss	3
III-A	Backward w.r.t. the probabilities	3
IV	Softmax	3
IV-A	The Jacobian	3
IV-B	Backward w.r.t. the logits	4
IV-C	Fused softmax + cross-entropy	4
V	Linear Layer (Matmul)	5
VI	GELU	6
VII	LayerNorm	7
VIII	Attention	7
IX	Embeddings (Encoder)	10
X	The Full Backward Pass	10
	Appendix A: Gradient Checking	10
	Appendix B: Useful Identities	10
	References	10

I. NOTATION AND CONVENTIONS

Shapes follow the kernel code, as collected in Table I.

Indices: b over batch, t over positions, i, j, k over channels or vocab entries. A row of activations at one position is $x_{bt} \in \mathbb{R}^C$.

a) *Why index form?*: Matrix calculus obscures backpropagation under transposes and Kronecker products. This document writes every relation in *index form* (element-by-element), reducing tensor operations to scalar calculus. Treating entries as independent scalar variables allows standard derivative rules. Matching indices then reassembles scalar equations back into matrix form.

b) *Linear algebra in index form.*: A matrix entry is named (row, column): A_{ij} is row i , column j . Three definitions cover every matrix manipulation in this document. The transpose swaps index roles:

$$(A^T)_{ij} = A_{ji}. \quad (1)$$

The matrix product contracts the inner indices:

$$(AB)_{ik} = \sum_j A_{ij} B_{jk}. \quad (2)$$

In code, this summation represents a loop over the shared dimension j :

```
val = 0.0
for j in range(width):
    val += A[i, j] * B[j, k]
```

Reversing (2) identifies matrix products. Summing over a shared index yields a product when the index is the left factor's column and the right factor's row. Elsewhere, a transpose (1) restores this alignment. This generates every transpose in the backward pass. Adding a vector to a matrix broadcasts the vector over rows:

$$(M + b)_{ni} = M_{ni} + b_i. \quad (3)$$

c) *Target and probe indices.*: Differentiating a tensor expression requires distinct names for the output (*target*) indices and the differentiation (*probe*) indices. Perturbing X_{mk} to find the change in Y_{ni} requires independent letters ($m, k \neq n, i$). Reusing indices would restrict the perturbation to matching coordinate pairs. Independent probe indices evaluate the derivative of any output coordinate with respect to any input coordinate.

d) *Indexed variables under differentiation.*: Entries of distinct tensors are unrelated; their cross-derivatives vanish. The derivative of a tensor entry with respect to another is 1 for identical coordinates and 0 otherwise. The Kronecker delta records this relation:

$$\delta_{ij} = \begin{cases} 1 & i = j, \\ 0 & i \neq j. \end{cases} \quad (4)$$

TABLE I
SYMBOLS AND THEIR CORRESPONDING KERNEL PARAMETERS.

Symbol	Code	Meaning
B	batch_size	batch size
T	seq_len	sequence length
C	channels	model width (embedding dim)
V	vocab_size	real vocabulary size
V_p	vocab_size_padded	padded vocabulary (row stride)
H	num_heads	attention heads, head size $h = C/H$

For a vector, one index pair determines equality; for a matrix, both row and column must match. A product of deltas encodes this conjunction:

$$\frac{\partial x_j}{\partial x_i} = \delta_{ji}, \quad \frac{\partial W_{ij}}{\partial W_{kl}} = \delta_{ik} \delta_{jl}. \quad (5)$$

Two concrete matrix instances:

$$\frac{\partial W_{12}}{\partial W_{12}} = \delta_{11} \delta_{22} = 1 \cdot 1 = 1, \quad \frac{\partial W_{12}}{\partial W_{21}} = \delta_{12} \delta_{21} = 0 \cdot 0 = 0. \quad (6)$$

Deltas exit derivations by collapsing sums. For fixed l , the Kronecker delta sifts the summation:

$$\sum_j \delta_{jl} A_j = A_l. \quad (7)$$

This sifting acts as a conditional check inside a loop:

```
sum_val = 0.0
for j in range(N):
    if j == l:
        sum_val += 1.0 * A[j]
    else:
        sum_val += 0.0 * A[j]
# sum_val is now exactly A[l]
```

Unsummed deltas remain in the final expression, enforcing index equality.

e) The backward convention.: Each operation receives the upstream gradient $\frac{\partial L}{\partial y}$ of the scalar loss L and computes the downstream gradient $\frac{\partial L}{\partial x}$ (or parameter gradient $\frac{\partial L}{\partial w}$) via the chain rule:

$$\frac{\partial L}{\partial x_i} = \sum_j \frac{\partial L}{\partial y_j} \frac{\partial y_j}{\partial x_i}. \quad (8)$$

This sum accumulates gradients from all output coordinates y_j that depend on x_i . Shared inputs or parameters (such as tied weights or broadcast biases) accumulate gradients across all active positions via in-place addition ($+=$).

f) Layout and transposes.: The *numerator layout* convention matches the shape of the gradient $\frac{\partial L}{\partial X}$ to the source tensor X . Index-level rearrangements then generate transposes automatically, avoiding memorized matrix identities.

g) Reading order.: While the forward pass runs encoder $\rightarrow$ layernorm $\rightarrow$ attention $\rightarrow$ matmul $\rightarrow$ GELU $\rightarrow$ softmax $\rightarrow$ cross-entropy, these derivations follow the backward flow. Starting at the loss (§III) and ending at the embeddings (§IX), each section consumes the upstream gradient from the previous section. §X reassembles the complete backward pass.

II. WARM-UP: SCALAR CHAIN RULE ON A TINY NETWORK

A two-step composition to fix conventions before the tensor ops: $z = wx + b$, $a = \sigma(z)$, $L = \frac{1}{2}(a - y)^2$.

Derivation. Backward through the loss, then the sigmoid:

$$\frac{\partial L}{\partial a} = a - y \quad (9)$$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \quad (10)$$

$$\frac{\partial a}{\partial z} = \sigma(z) \cdot (1 - \sigma(z)) \quad (11)$$

and since $a = \sigma(z)$,

$$\frac{\partial L}{\partial z} = (a - y) \cdot \frac{\partial a}{\partial z} = (a - y) \cdot a \cdot (1 - a). \quad (12)$$

For the weight:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w} \quad (13)$$

$$\frac{\partial z}{\partial w} = x \quad (14)$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x = (a - y) \cdot a \cdot (1 - a) \cdot x. \quad (15)$$

For the bias:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial b} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial b} \quad (16)$$

$$\frac{\partial z}{\partial b} = 1 \quad (17)$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} = (a - y) \cdot a \cdot (1 - a). \quad (18)$$

For the input:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial x} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x} \quad (19)$$

$$\frac{\partial z}{\partial x} = w \quad (20)$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot w = (a - y) \cdot a \cdot (1 - a) \cdot w. \quad (21)$$

Result. One chain back through the network, fanning out at the affine inputs:

$$\frac{\partial L}{\partial a} \rightarrow \frac{\partial L}{\partial z} \rightarrow \left\{ \frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}, \frac{\partial L}{\partial x} \right\} \quad (22)$$

with $\frac{\partial L}{\partial z} = (a - y) \cdot a \cdot (1 - a)$ computed once and reused: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x$, $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}$, $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot w$.

III. CROSS-ENTROPY LOSS

Standard cross-entropy between a target distribution $y_{bt} \in \mathbb{R}^V$ and predicted probabilities $p_{bt} \in \mathbb{R}^V$:

$$L_{bt} = - \sum_{i < V} y_{bt,i} \log p_{bt,i}. \quad (23)$$

For language modeling the target distribution is one-hot. Writing $\mathbb{K}[P]$ for the indicator of a condition P ,

$$\mathbb{K}[P] = \begin{cases} 1 & P \text{ is true,} \\ 0 & P \text{ is false,} \end{cases} \quad (24)$$

(the Kronecker delta (4) is this indicator specialized to an index equality, $\delta_{ij} = \mathbb{K}[i = j]$), the target with token $y_{bt} \in \{0, \dots, V-1\}$ is $y_{bt,i} = \mathbb{K}[i = y_{bt}]$: all probability mass on the target token, zero elsewhere. Every term of (23) but one then vanishes, and the loss collapses to the form computed by `crossentropy_ohw_fwd`:

$$L_{bt} = - \log p_{bt, y_{bt}}. \quad (25)$$

A. Backward w.r.t. the probabilities

Derivation. Index roles: i names the probed coordinate of p_{bt} , j the summed one. Rows are independent, and within a row the coordinates of p_{bt} are independent variables obeying (5). Break the standard form (23) out term by term:

$$L_{bt} = - (y_{bt,0} \log p_{bt,0} + y_{bt,1} \log p_{bt,1} + y_{bt,2} \log p_{bt,2} + \dots + y_{bt,V-1} \log p_{bt,V-1}). \quad (26)$$

Differentiating with respect to one coordinate $p_{bt,i}$, every term of the broken-out sum except the i -th is constant, so only its derivative survives:

$$\begin{aligned} \frac{\partial L}{\partial p_{bt,i}} &= - \frac{\partial}{\partial p_{bt,i}} (y_{bt,0} \log p_{bt,0} + \dots + y_{bt,i} \log p_{bt,i} \\ &\quad + \dots + y_{bt,V-1} \log p_{bt,V-1}) \\ &= - y_{bt,i} \frac{\partial}{\partial p_{bt,i}} \log p_{bt,i}. \end{aligned} \quad (27)$$

The surviving factor is the derivative of the logarithm,

$$\frac{d}{dx} \log x = \frac{1}{x} \implies \frac{\partial}{\partial p_{bt,i}} \log p_{bt,i} = \frac{1}{p_{bt,i}}, \quad (28)$$

In delta form the same two steps are (5) followed by the sift (7): each term of the sum differentiates to $(y_{bt,j}/p_{bt,j}) \delta_{ji}$, and the sum collapses to the single term $j = i$,

$$\frac{\partial L}{\partial p_{bt,i}} = - \sum_{j < V} \frac{y_{bt,j}}{p_{bt,j}} \delta_{ji} = - \frac{y_{bt,i}}{p_{bt,i}}, \quad (29)$$

giving the standard-sum gradient, valid for *any* target distribution y_{bt} :

$$\frac{\partial L}{\partial p_{bt,i}} = - \frac{\partial}{\partial p_{bt,i}} \sum_{j < V} y_{bt,j} \log p_{bt,j} = - \frac{y_{bt,i}}{p_{bt,i}}. \quad (30)$$

Substituting the one-hot target $y_{bt,i} = \mathbb{K}[i = y_{bt}]$, (25) depends only on the target coordinate $p_{bt, y_{bt}}$:

$$\frac{\partial L}{\partial p_{bt,i}} = \begin{cases} -\frac{1}{p_{bt, y_{bt}}} & i = y_{bt}, \\ 0 & \text{otherwise.} \end{cases} \quad (31)$$

Result.

$$\frac{\partial L}{\partial p_{bt,i}} = - \frac{\mathbb{K}[i = y_{bt}]}{p_{bt,i}}. \quad (32)$$

IV. SOFTMAX

Forward: from logits $\ell_{bt} \in \mathbb{R}^V$ (real entries $i < V$),

$$p_{bt,i} = \frac{e^{\ell_{bt,i} - m_{bt}}}{\sum_{j < V} e^{\ell_{bt,j} - m_{bt}}}, \quad m_{bt} = \max_{j < V} \ell_{bt,j}. \quad (33)$$

A. The Jacobian

Row indices bt are dropped throughout: each row is an independent softmax, so the Jacobian is per-row.

Derivation. The object here is the Jacobian entry $\frac{\partial p_i}{\partial \ell_j}$ alone: how one probability moves with one logit. Index roles: i names the probed probability, j the probed logit, and the logits of a row are independent variables obeying (5), $\frac{\partial \ell_a}{\partial \ell_j} = \delta_{aj}$. The loss enters only in the subsections that follow, where an upstream gradient is chained through these entries. Before differentiating: the max-subtraction in (33) cancels, because for any constant c

$$\frac{e^{\ell_i - c}}{\sum_{j < V} e^{\ell_j - c}} = \frac{e^{-c} e^{\ell_i}}{e^{-c} \sum_{j < V} e^{\ell_j}} = \frac{e^{\ell_i}}{\sum_{j < V} e^{\ell_j}}, \quad (34)$$

so m_{bt} does not change the derivative and the unshifted form can be differentiated. Break the denominator sum out in the same format:

$$p_i = \frac{e^{\ell_i}}{S}, \quad S = e^{\ell_0} + e^{\ell_1} + e^{\ell_2} + \dots + e^{\ell_{V-1}}. \quad (35)$$

Every entry comes from the quotient rule: for u/v and a scalar argument x ,

$$\frac{\partial}{\partial x} \frac{u}{v} = \frac{\frac{\partial u}{\partial x} v - u \frac{\partial v}{\partial x}}{v^2}. \quad (36)$$

Work the 2×2 block of entries in p_1, p_2 and ℓ_1, ℓ_2 explicitly before generalizing.

a) Entry $\frac{\partial p_1}{\partial \ell_1}$: Here $u = e^{\ell_1}$ and $v = S$, and *both* move with ℓ_1 :

$$\frac{\partial u}{\partial \ell_1} = \frac{\partial}{\partial \ell_1} e^{\ell_1} = e^{\ell_1}, \quad (37)$$

$$\frac{\partial v}{\partial \ell_1} = \frac{\partial}{\partial \ell_1} (e^{\ell_0} + e^{\ell_1} + e^{\ell_2} + \dots + e^{\ell_{V-1}}) = e^{\ell_1}, \quad (38)$$

every term of the broken-out sum except e^{ℓ_1} being constant. Substituting (37) and (38) into the quotient rule (36) with the values filled in:

$$\begin{aligned} \frac{\partial p_1}{\partial \ell_1} &= \frac{e^{\ell_1} (e^{\ell_0} + e^{\ell_1} + \dots + e^{\ell_{V-1}}) - e^{\ell_1} e^{\ell_1}}{(e^{\ell_0} + e^{\ell_1} + \dots + e^{\ell_{V-1}})^2} \\ &= \frac{e^{\ell_1} S - e^{\ell_1} e^{\ell_1}}{S^2} = \frac{e^{\ell_1}}{S} - \left(\frac{e^{\ell_1}}{S} \right)^2 \\ &= p_1 - p_1^2 = p_1 (1 - p_1). \end{aligned} \quad (39)$$

b) *Entry* $\frac{\partial p_1}{\partial \ell_2}$: Now the numerator does *not* move: $u = e^{\ell_1}$ is constant in ℓ_2 , so $\frac{\partial u}{\partial \ell_2} = 0$, while again exactly one term of the broken-out denominator survives:

$$\frac{\partial v}{\partial \ell_2} = \frac{\partial}{\partial \ell_2} (e^{\ell_0} + e^{\ell_1} + e^{\ell_2} + \dots + e^{\ell_{V-1}}) = e^{\ell_2}. \quad (40)$$

Substituting into (36) with the values filled in:

$$\begin{aligned} \frac{\partial p_1}{\partial \ell_2} &= \frac{0 \cdot (e^{\ell_0} + e^{\ell_1} + \dots + e^{\ell_{V-1}}) - e^{\ell_1} e^{\ell_2}}{(e^{\ell_0} + e^{\ell_1} + \dots + e^{\ell_{V-1}})^2} \\ &= -\frac{e^{\ell_1}}{S} \cdot \frac{e^{\ell_2}}{S} = -p_1 p_2. \end{aligned} \quad (41)$$

c) *Entries* $\frac{\partial p_2}{\partial \ell_1}$ and $\frac{\partial p_2}{\partial \ell_2}$: The same two computations with the indices swapped. For $\frac{\partial p_2}{\partial \ell_1}$: $u = e^{\ell_2}$ is constant in ℓ_1 and $\frac{\partial v}{\partial \ell_1} = e^{\ell_1}$ by (38); for $\frac{\partial p_2}{\partial \ell_2}$ both numerator and denominator move, exactly as in (39):

$$\frac{\partial p_2}{\partial \ell_1} = \frac{0 \cdot S - e^{\ell_2} e^{\ell_1}}{S^2} = -p_2 p_1, \quad (42)$$

$$\frac{\partial p_2}{\partial \ell_2} = \frac{e^{\ell_2} S - e^{\ell_2} e^{\ell_2}}{S^2} = p_2 (1 - p_2). \quad (43)$$

d) *The pattern*: Arranged as a block of the Jacobian, the four entries are

$$\begin{pmatrix} \frac{\partial p_1}{\partial \ell_1} & \frac{\partial p_1}{\partial \ell_2} \\ \frac{\partial p_2}{\partial \ell_1} & \frac{\partial p_2}{\partial \ell_2} \end{pmatrix} = \begin{pmatrix} p_1(1-p_1) & -p_1 p_2 \\ -p_2 p_1 & p_2(1-p_2) \end{pmatrix}: \quad (44)$$

diagonal entries $p_i(1-p_i)$, off-diagonal entries $-p_i p_j$. The general entry repeats the same quotient rule. Each v -derivative above is one instance of the general fact that exactly one term of the broken-out S involves ℓ_j . In delta form, each term differentiates by (5) and the sum sifts (7):

$$\frac{\partial S}{\partial \ell_j} = \sum_{a < V} \frac{\partial}{\partial \ell_j} e^{\ell_a} = \sum_{a < V} e^{\ell_a} \delta_{aj} = e^{\ell_j}. \quad (45)$$

The diagonal case $j = i$ repeats (39) (the numerator also moves):

$$\begin{aligned} \frac{\partial p_i}{\partial \ell_i} &= \frac{e^{\ell_i} S - e^{\ell_i} e^{\ell_i}}{S^2} = \frac{e^{\ell_i}}{S} - \left(\frac{e^{\ell_i}}{S}\right)^2 \\ &= p_i - p_i^2 = p_i(1-p_i). \end{aligned} \quad (46)$$

The off-diagonal case $j \neq i$ repeats (41) (the numerator e^{ℓ_i} is constant in ℓ_j):

$$\frac{\partial p_i}{\partial \ell_j} = \frac{0 \cdot S - e^{\ell_i} e^{\ell_j}}{S^2} = -p_i p_j. \quad (47)$$

Result. The two cases combine into one expression via the Kronecker delta (4), which selects the diagonal case:

$$\frac{\partial p_i}{\partial \ell_j} = p_i (\delta_{ij} - p_j). \quad (48)$$

Check against (46) and (47): $j = i$ gives $p_i(1-p_i)$, and $j \neq i$ gives $-p_i p_j$.

B. Backward w.r.t. the logits

Derivation. The upstream gradient $\frac{\partial L}{\partial p}$ enters through the chain rule (8) summed over the row, one full column of the Jacobian (48) per output i . The summed index j runs over the upstream coordinates; the free index i names the logit gradient entry being produced. Broken out:

$$\begin{aligned} \frac{\partial L}{\partial \ell_i} &= \sum_{j < V} \frac{\partial L}{\partial p_j} \frac{\partial p_j}{\partial \ell_i} \\ &= \frac{\partial L}{\partial p_0} \frac{\partial p_0}{\partial \ell_i} + \frac{\partial L}{\partial p_1} \frac{\partial p_1}{\partial \ell_i} + \frac{\partial L}{\partial p_2} \frac{\partial p_2}{\partial \ell_i} \\ &\quad + \dots + \frac{\partial L}{\partial p_{V-1}} \frac{\partial p_{V-1}}{\partial \ell_i}. \end{aligned} \quad (49)$$

Substitute (46) and (47); the $j = i$ term is the diagonal case, every other term is off-diagonal:

$$\begin{aligned} \frac{\partial L}{\partial \ell_i} &= \frac{\partial L}{\partial p_i} p_i (1-p_i) + \sum_{j \neq i} \frac{\partial L}{\partial p_j} (-p_j p_i) \\ &= p_i \frac{\partial L}{\partial p_i} - p_i p_i \frac{\partial L}{\partial p_i} - p_i \sum_{j \neq i} \frac{\partial L}{\partial p_j} p_j. \end{aligned} \quad (50)$$

The middle term is exactly the missing $j = i$ element of the sum, so it re-absorbs:

$$\frac{\partial L}{\partial \ell_i} = p_i \frac{\partial L}{\partial p_i} - p_i \sum_{j < V} \frac{\partial L}{\partial p_j} p_j, \quad (51)$$

where the remaining sum is the same one for every i in the row. Broken out in the same format, it is

$$\begin{aligned} \sum_{j < V} \frac{\partial L}{\partial p_j} p_j &= \frac{\partial L}{\partial p_0} p_0 + \frac{\partial L}{\partial p_1} p_1 \\ &\quad + \frac{\partial L}{\partial p_2} p_2 + \dots + \frac{\partial L}{\partial p_{V-1}} p_{V-1}. \end{aligned} \quad (52)$$

Result.

$$\frac{\partial L}{\partial \ell_i} = p_i \left(\frac{\partial L}{\partial p_i} - \sum_{j < V} \frac{\partial L}{\partial p_j} p_j \right), \quad i < V, \quad (53)$$

and $\frac{\partial L}{\partial \ell_i} = 0$ for the padded region $i \geq V$. Summing (53) over the row, the leading p_i distributes into both terms; the probabilities multiplying the (constant) dot product sum to one, leaving two copies of the same dot product:

$$\begin{aligned} \sum_{i < V} \frac{\partial L}{\partial \ell_i} &= \sum_{i < V} p_i \frac{\partial L}{\partial p_i} - \left(\sum_{i < V} p_i \right) \sum_{j < V} \frac{\partial L}{\partial p_j} p_j \\ &= \sum_{i < V} p_i \frac{\partial L}{\partial p_i} - \sum_{j < V} \frac{\partial L}{\partial p_j} p_j = 0. \end{aligned} \quad (54)$$

The gradient row always sums to zero, consistent with the shift invariance (34).

C. Fused softmax + cross-entropy

The composition is the case that matters for the model: backprop through (25) and (33) together and watch the Jacobian collapse.

Derivation. Write out the chain per logit coordinate, full indices restored: the summed index i runs over the probabilities, and the free index j names the logit gradient entry being produced:

$$\frac{\partial L}{\partial \ell_{bt,j}} = \sum_{i < V} \frac{\partial L}{\partial p_{bt,i}} \frac{\partial p_{bt,i}}{\partial \ell_{bt,j}}. \quad (55)$$

Substitute what is already known into the combined equation: the cross-entropy gradient (32) for the first factor and the Jacobian (48) for the second:

$$\frac{\partial L}{\partial \ell_{bt,j}} = \sum_{i < V} \left(-\frac{\mathbb{K}[i = y_{bt}]}{p_{bt,i}} \right) p_{bt,i} (\delta_{ij} - p_{bt,j}) \quad (56)$$

The $p_{bt,i}$ in the denominator cancels against the $p_{bt,i}$ the Jacobian contributes:

$$= - \sum_{i < V} \mathbb{K}[i = y_{bt}] (\delta_{ij} - p_{bt,j}) \quad (57)$$

and the one-hot indicator keeps exactly one term of the sum, $i = y_{bt}$:

$$= -(\delta_{y_{bt}j} - p_{bt,j}) = p_{bt,j} - \mathbb{K}[j = y_{bt}]. \quad (58)$$

Result.

$$\frac{\partial L}{\partial \ell_{bt,j}} = p_{bt,j} - \mathbb{K}[j = y_{bt}], \quad j < V, \quad (59)$$

and $\frac{\partial L}{\partial \ell_{bt,j}} = 0$ for the padded region $j \geq V$. The Jacobian collapsed entirely: no sum over the row survives in (59).

Cross-check via the general backward (53): with the cross-entropy upstream (32), the dot product evaluates to

$$\sum_{j < V} \frac{\partial L}{\partial p_{bt,j}} p_{bt,j} = \sum_{j < V} \left(-\frac{\mathbb{K}[j = y_{bt}]}{p_{bt,j}} \right) p_{bt,j} = -1, \quad (60)$$

a constant: for a one-hot target the row-wide weighted sum is known before reading a single element. Substituting -1 for the dot product in (53), the $p_{bt,i}$ outside the parentheses cancels the reciprocal inside them:

$$\begin{aligned} \frac{\partial L}{\partial \ell_{bt,i}} &= p_{bt,i} \left(\frac{\partial L}{\partial p_{bt,i}} - (-1) \right) \\ &= p_{bt,i} \left(-\frac{\mathbb{K}[i = y_{bt}]}{p_{bt,i}} + 1 \right) \\ &= p_{bt,i} - \mathbb{K}[i = y_{bt}], \end{aligned} \quad (61)$$

agreeing with (59): chaining through the Jacobian directly and substituting into the general backward are two routes to the same gradient. The zero-sum check of §IV also holds: the probabilities sum to one, and the indicator fires exactly once per row:

$$\sum_{j < V} (p_{bt,j} - \mathbb{K}[j = y_{bt}]) = 1 - 1 = 0. \quad (62)$$

The collapse did not need one-hotness. It needed only the standard-sum gradient (30) and the fact that the target is a

distribution, $\sum_{i < V} y_{bt,i} = 1$. Repeating (55) with a general target distribution,

$$\begin{aligned} \frac{\partial L}{\partial \ell_{bt,j}} &= \sum_{i < V} \left(-\frac{y_{bt,i}}{p_{bt,i}} \right) p_{bt,i} (\delta_{ij} - p_{bt,j}) \\ &= -y_{bt,j} + p_{bt,j} \sum_{i < V} y_{bt,i} = p_{bt,j} - y_{bt,j}, \end{aligned} \quad (63)$$

the one-hot result (59) being the special case $y_{bt,j} = \mathbb{K}[j = y_{bt}]$. Softmax + cross-entropy always backpropagates as *predicted minus target*.

V. LINEAR LAYER (MATMUL)

Forward: $Y = XW^\top + b$ with $X \in \mathbb{R}^{(B \cdot T) \times C_{in}}$, $W \in \mathbb{R}^{C_{out} \times C_{in}}$, $b \in \mathbb{R}^{C_{out}}$.

Derivation. Index form throughout: n runs over the $N = B \cdot T$ flattened rows ($n = bt$), j over C_{in} , and i over C_{out} . Read every entry as (row, column): X_{nj} is (position n , in-channel j); W_{ij} is (out-channel i , in-channel j); Y_{ni} is (position n , out-channel i); b_i has the single index (out-channel i).

Differentiation needs a *second*, independent set of index names. Asking how Y_{ni} moves when one entry of X moves must allow the moving entry to be any entry, so the probe entry gets fresh indices: X_{mk} (position m , in-channel k), and likewise W_{kl} (out-channel k , in-channel l) and b_k . Keeping the target indices (n, i) and the probe indices distinct asks every combination of which-output against which-input at once.

a) *The transpose under differentiation:* By the transpose definition (1), $(W^\top)_{ji} = W_{ij}$: the same independent variables, relabeled. Applying (5) through the relabeling, the transpose changes nothing except which of W 's indices is compared with which probe index:

$$\frac{\partial (W^\top)_{ji}}{\partial W_{kl}} = \frac{\partial W_{ij}}{\partial W_{kl}} = \delta_{ik} \delta_{jl}. \quad (64)$$

With (1) absorbed and the bias broadcast (3) written out, the forward in indices is

$$\begin{aligned} Y_{ni} &= \sum_{j < C_{in}} X_{nj} (W^\top)_{ji} + b_i = \sum_{j < C_{in}} X_{nj} W_{ij} + b_i \\ &= X_{n0} W_{i0} + X_{n1} W_{i1} + \dots + X_{n, C_{in}-1} W_{i, C_{in}-1} + b_i. \end{aligned} \quad (65)$$

b) *Local derivative $\frac{\partial Y}{\partial X}$:* Target Y_{ni} (position n , out-channel i); probe X_{mk} (position m , in-channel k): four free indices, every pair of output entry against input entry at once. Differentiate the broken-out form (65) term by term. W and b are constants here, and each X_{nj} obeys (5):

$$\begin{aligned} \frac{\partial Y_{ni}}{\partial X_{mk}} &= \sum_{j < C_{in}} W_{ij} \frac{\partial X_{nj}}{\partial X_{mk}} = \sum_{j < C_{in}} W_{ij} \delta_{nm} \delta_{jk} \\ &= \delta_{nm} \sum_{j < C_{in}} W_{ij} \delta_{jk} = \delta_{nm} W_{ik}. \end{aligned} \quad (66)$$

The row delta δ_{nm} contains no j , so it factors out of the sum; the column delta sifts the sum down to $j = k$ by (7). δ_{nm} survives because neither n nor m is summed here: it waits for the chain rule, recording that row n of Y reads only row $m = n$ of X .

c) *Local derivative* $\frac{\partial Y}{\partial W}$: Target Y_{ni} , probe W_{kl} (out-channel k , in-channel l). Now X is the constant and each W_{ij} obeys (64):

$$\begin{aligned} \frac{\partial Y_{ni}}{\partial W_{kl}} &= \sum_{j < C_{in}} X_{nj} \frac{\partial W_{ij}}{\partial W_{kl}} = \sum_{j < C_{in}} X_{nj} \delta_{ik} \delta_{jl} \\ &= \delta_{ik} \sum_{j < C_{in}} X_{nj} \delta_{jl} = \delta_{ik} X_{nl}. \end{aligned} \quad (67)$$

The same shape with the roles swapped: the out-channel delta δ_{ik} has no j and factors out unconsumed, while the in-channel delta sifts j down to l .

d) *Local derivative* $\frac{\partial Y}{\partial b}$, and its gotcha: The bias term in (65) is a bare b_i , and b is a vector, so (5) needs only one index pair:

$$\frac{\partial Y_{ni}}{\partial b_k} = \frac{\partial b_i}{\partial b_k} = \delta_{ik}, \quad (68)$$

with no dependence on n at all. That is the broadcast: the same b_k appears in every one of the N rows of Y , so b_k has N distinct output coordinates moving with it. The local derivative looks as innocent as the scalar warm-up (§II, $\frac{\partial z}{\partial b} = 1$). That is exactly what makes it a trap. The fan-out is invisible here. It reappears only in the chain rule, as a sum that is easy to forget.

e) *Chain rule*: L depends on every entry of Y , so (8) sums over both of the *output's* indices (n, i). The probe indices are the free ones: they name which gradient entry is being produced.

$$\frac{\partial L}{\partial \theta} = \sum_{n < N} \sum_{i < C_{out}} \frac{\partial L}{\partial Y_{ni}} \frac{\partial Y_{ni}}{\partial \theta}. \quad (69)$$

For the input, substitute (66). The surviving row delta now meets the n sum and sifts it down to $n = m$; nothing constrains i , so the i sum remains as the contraction with W :

$$\begin{aligned} \frac{\partial L}{\partial X_{mk}} &= \sum_{n < N} \sum_{i < C_{out}} \frac{\partial L}{\partial Y_{ni}} \delta_{nm} W_{ik} \\ &= \sum_{i < C_{out}} \frac{\partial L}{\partial Y_{mi}} W_{ik} = \left(\frac{\partial L}{\partial Y} W \right)_{mk}, \end{aligned} \quad (70)$$

free indices (m, k) on both sides: row m , in-channel k of the input gradient. The last step is the recognition rule of (2): the summed i sits as the column of $\frac{\partial L}{\partial Y}$ (entry $\frac{\partial L}{\partial Y_{mi}}$) and the row of W (entry W_{ik}), so the sum is a plain product with no transpose. For the weight, substitute (67). The out-channel delta sifts the i sum down to $i = k$, but nothing constrains n , so the n sum *survives*:

$$\begin{aligned} \frac{\partial L}{\partial W_{kl}} &= \sum_{n < N} \sum_{i < C_{out}} \frac{\partial L}{\partial Y_{ni}} \delta_{ik} X_{nl} \\ &= \sum_{n < N} \frac{\partial L}{\partial Y_{nk}} X_{nl} = \left(\frac{\partial L}{\partial Y}^\top X \right)_{kl}, \end{aligned} \quad (71)$$

free indices (k, l): out-channel k , in-channel l , the entry layout of W itself. Here the recognition rule of (2) forces a transpose: the summed n sits as the *row* of both $\frac{\partial L}{\partial Y}$ and X . Transposing the left factor moves it, $(\frac{\partial L}{\partial Y}^\top)_{kn} = \frac{\partial L}{\partial Y_{nk}}$ by (1), and the sum becomes a product. For the bias, substitute (68). The delta

sifts the i sum; the local derivative never mentioned n , so the whole n sum survives. Broken out:

$$\begin{aligned} \frac{\partial L}{\partial b_k} &= \sum_{n < N} \sum_{i < C_{out}} \frac{\partial L}{\partial Y_{ni}} \delta_{ik} = \sum_{n < N} \frac{\partial L}{\partial Y_{nk}} \\ &= \frac{\partial L}{\partial Y_{0k}} + \frac{\partial L}{\partial Y_{1k}} + \frac{\partial L}{\partial Y_{2k}} + \cdots + \frac{\partial L}{\partial Y_{N-1,k}}. \end{aligned} \quad (72)$$

This is the gotcha of (68) coming due: the local derivative was a lone δ_{ik} , but the gradient is a full reduction over all $B \cdot T$ rows. In the scalar warm-up the same formula holds with $N = 1$, which is why $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}$ there: the sum was present all along, with one term.

Result.

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W, \quad \frac{\partial L}{\partial W} = \frac{\partial L}{\partial Y}^\top X, \quad \frac{\partial L}{\partial b_k} = \sum_{n < N} \frac{\partial L}{\partial Y_{nk}}. \quad (73)$$

Shape checks, with $\frac{\partial L}{\partial Y} \in \mathbb{R}^{N \times C_{out}}$: $(N \times C_{out})(C_{out} \times C_{in}) = N \times C_{in}$ for $\frac{\partial L}{\partial X}$; $(C_{out} \times N)(N \times C_{in}) = C_{out} \times C_{in}$ for $\frac{\partial L}{\partial W}$; $\frac{\partial L}{\partial b} \in \mathbb{R}^{C_{out}}$.

The reduction over $B \cdot T$ appears in $\frac{\partial L}{\partial W}$ and $\frac{\partial L}{\partial b}$ (every position contributes to the same shared parameter: the $+$ = convention of §I) and not in $\frac{\partial L}{\partial X}$ (each row of the input gets its own gradient row).

Transpose bookkeeping: the forward carries the transpose ($Y = XW^\top$), and two consequences follow. The input gradient needs *no* transpose ($\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W$, not $\frac{\partial L}{\partial Y} W^\top$). And $\frac{\partial L}{\partial W}$ comes out directly in W 's own ($C_{out} \times C_{in}$) layout, ready for the optimizer. Both facts fall out of (64) rather than being conventions to memorize.

VI. GELU

Forward (tanh approximation, as in llm.c):

$$\text{GELU}(x) = \frac{1}{2} x \left(1 + \tanh \left[\sqrt{2/\pi} (x + 0.044715 x^3) \right] \right). \quad (74)$$

Name the inner argument once; everything chains through it:

$$\begin{aligned} u(x) &= \sqrt{2/\pi} (x + 0.044715 x^3), \\ \text{GELU}(x) &= \frac{1}{2} x (1 + \tanh u). \end{aligned} \quad (75)$$

Derivation. The object is the scalar derivative $\text{GELU}'(x)$. How it enters the chain rule is recorded in the result.

a) *Product rule on the definition*: (75) is a product of $\frac{1}{2}x$ and $(1 + \tanh u)$:

$$\text{GELU}'(x) = \frac{1}{2} (1 + \tanh u) + \frac{1}{2} x \frac{\partial}{\partial x} \tanh u. \quad (76)$$

b) *The tanh derivative via sech²*: The derivative of $\tanh$ is sech^2 , and the inner function chains through it:

$$\frac{\partial}{\partial x} \tanh u = \text{sech}^2(u) u'(x), \quad (77)$$

with the polynomial derivative of the inner argument

$$\begin{aligned} u'(x) &= \sqrt{2/\pi} (1 + 3 \cdot 0.044715 x^2) \\ &= \sqrt{2/\pi} (1 + 0.134145 x^2). \end{aligned} \quad (78)$$

c) *Converting sech^2 to $1 - \tanh^2$* : Divide the hyperbolic Pythagorean identity $\cosh^2 u - \sinh^2 u = 1$ through by $\cosh^2 u$:

$$1 - \tanh^2 u = \text{sech}^2 u, \quad (79)$$

so (77) needs no separate sech evaluation:

$$\frac{\partial}{\partial x} \tanh u = (1 - \tanh^2 u) u'(x). \quad (80)$$

d) *The derivative rolls into the definition*: $\tanh$ differentiates into a function of itself, $\tanh' = 1 - \tanh^2$, and $\tanh u$ already sits inside the forward (75). Solving the definition for it (for $x \neq 0$):

$$1 + \tanh u = \frac{2 \text{GELU}(x)}{x} \implies \tanh u = \frac{2 \text{GELU}(x)}{x} - 1. \quad (81)$$

Substituting (80) into (76), every quantity the derivative needs is either x itself or the $\tanh u$ the forward already computed:

$$\begin{aligned} \text{GELU}'(x) &= \frac{1}{2}(1 + \tanh u) + \frac{1}{2}x(1 - \tanh^2 u)u'(x) \\ &= \frac{\text{GELU}(x)}{x} + \frac{1}{2}x(1 - \tanh^2 u)u'(x). \end{aligned} \quad (82)$$

Result.

$$\text{GELU}'(x) = \frac{1}{2}(1 + \tanh u) + \frac{1}{2}x(1 - \tanh^2 u)u'(x), \quad (83)$$

with u and u' from (75) and (78). One evaluation of $\tanh u$ serves both terms: the self-reference $\tanh' = 1 - \tanh^2$ (79) replaces sech , and via (81) the first term is the forward output divided by x . At $x = 0$ use (83) directly: $u(0) = 0$ and $\tanh 0 = 0$ give $\text{GELU}'(0) = \frac{1}{2}$ (the $\text{GELU}(x)/x$ form of (82) is singular there).

GELU is elementwise: one output coordinate moves with exactly one input coordinate. By (5) the Jacobian is diagonal,

$$\frac{\partial \text{GELU}(x_a)}{\partial x_b} = \text{GELU}'(x_a) \delta_{ab}, \quad (84)$$

and the chain rule (8) sifts (7) to a pointwise product: $\frac{\partial L}{\partial y_a} = \frac{\partial L}{\partial x_a} \text{GELU}'(x_a)$.

VII. LAYERNORM

Forward, per position $x \in \mathbb{R}^C$ with scale γ and shift β :

$$\begin{aligned} \mu &= \frac{1}{C} \sum_j x_j, & \sigma^2 &= \frac{1}{C} \sum_j (x_j - \mu)^2 + \varepsilon, \\ \hat{x}_i &= \frac{x_i - \mu}{\sqrt{\sigma^2}}, \\ y_i &= \gamma_i \hat{x}_i + \beta_i. \end{aligned} \quad (85)$$

Derivation. We compute the partial derivatives of the output y_i with respect to its parameters and inputs. Note that y_i depends on every x_j in the row through the statistics μ and σ^2 .

a) *Parameter partials*: The output y_i depends only on its own channel's scale and shift:

$$\frac{\partial y_i}{\partial \gamma_i} = \frac{x_i - \mu}{\sqrt{\sigma^2}}, \quad \frac{\partial y_i}{\partial \beta_i} = 1. \quad (86)$$

b) *Input partials*: To find the Jacobian $\frac{\partial y_j}{\partial x_i}$, we first differentiate the row statistics. Each x_i contributes $1/C$ to the mean and $2(x_i - \mu)/C$ to the variance:

$$\frac{\partial \mu}{\partial x_i} = \frac{1}{C}, \quad \frac{\partial \sigma^2}{\partial x_i} = \frac{2(x_i - \mu)}{C}. \quad (87)$$

Now differentiate the normalized value $\hat{x}_j$ with respect to x_i using the quotient rule on $\hat{x}_j = (x_j - \mu)(\sigma^2)^{-1/2}$:

$$\begin{aligned} \frac{\partial \hat{x}_j}{\partial x_i} &= \frac{\partial}{\partial x_i} \left(\frac{x_j - \mu}{\sqrt{\sigma^2}} \right) \\ &= \frac{(\delta_{ji} - \frac{1}{C})\sqrt{\sigma^2} - (x_j - \mu)\frac{1}{2}(\sigma^2)^{-1/2}\frac{\partial \sigma^2}{\partial x_i}}{\sigma^2} \\ &= \frac{1}{\sqrt{\sigma^2}} \left(\delta_{ji} - \frac{1}{C} - \frac{(x_j - \mu)(x_i - \mu)}{C\sigma^2} \right). \end{aligned} \quad (88)$$

The partial $\frac{\partial y_j}{\partial x_i} = \gamma_j \frac{\partial \hat{x}_j}{\partial x_i}$ records how every output in the row is affected by a change in x_i .

Result. The total gradient for each quantity is the sum of its contributions to the loss through every output y_j , substituting the partials derived above:

$$\frac{\partial L}{\partial \gamma_i} = \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial \gamma_i} = \frac{\partial L}{\partial y_i} \frac{x_i - \mu}{\sqrt{\sigma^2}}, \quad (89)$$

$$\frac{\partial L}{\partial \beta_i} = \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial \beta_i} = \frac{\partial L}{\partial y_i}. \quad (90)$$

For the input x_i , we sum over the entire row $j \in \{0, \dots, C-1\}$. Expanding the Kronecker delta δ_{ji} into its indicator form $\mathbb{K}[j = i]$ to clarify the sifting:

$$\begin{aligned} \frac{\partial L}{\partial x_i} &= \sum_j \frac{\partial L}{\partial y_j} \frac{\partial y_j}{\partial x_i} \\ &= \sum_j \frac{\partial L}{\partial y_j} \left[\frac{\gamma_j}{\sqrt{\sigma^2}} \left(\delta_{ji} - \frac{1}{C} - \frac{\hat{x}_i \hat{x}_j}{C} \right) \right] \\ &= \frac{1}{\sqrt{\sigma^2}} \left(\sum_j \gamma_j \frac{\partial L}{\partial y_j} \mathbb{K}[j = i] - \frac{1}{C} \sum_j \gamma_j \frac{\partial L}{\partial y_j} \right. \\ &\quad \left. - \frac{\hat{x}_i}{C} \sum_j \gamma_j \frac{\partial L}{\partial y_j} \hat{x}_j \right) \\ &= \frac{1}{\sqrt{\sigma^2}} \left(\gamma_i \frac{\partial L}{\partial y_i} - \frac{1}{C} \sum_j \gamma_j \frac{\partial L}{\partial y_j} \right. \\ &\quad \left. - \frac{\hat{x}_i}{C} \sum_j \gamma_j \frac{\partial L}{\partial y_j} \hat{x}_j \right). \end{aligned} \quad (91)$$

The row-wide sums $\sum \gamma_j \frac{\partial L}{\partial y_j} / C$ and $\sum \gamma_j \frac{\partial L}{\partial y_j} \hat{x}_j / C$ match the row statistics s_1 and s_2 computed in the first pass of `layernorm_bwd`.

VIII. ATTENTION

Forward, per head: $A = \text{softmax}(QK^\top / \sqrt{h} + M)$, $Y = AV$, with causal mask M and $Q = XW_Q$, etc.

Derivation. Index form throughout: i runs over query positions T ($i < T$), j over key/value positions T ($j < T$), and d

over the head dimension h ($d < h$). Batch and head indices are dropped throughout: each head is independent, so the gradients are derived per-head. The channel size of the query and key projection within each head is the head dimension h , which corresponds to the standard notation d_k used in the literature (scaling the attention scores by $1/\sqrt{d_k}$). The head dimension is obtained as $h = C/H$, where C is the total channel width (embedding dimension) and H is the number of attention heads (see Table I in §I). The scaling factor is written as $1/\sqrt{h}$ to match the head dimension symbol in the kernel code. Read every entry as (row, column): Q_{id} is (query position i , channel d); K_{jd} is (key position j , channel d); V_{jd} is (value position j , channel d); Y_{id} is (output position i , channel d); S_{ij} is the raw score at (query position i , key position j); A_{ij} is the attention probability at (query position i , key position j).

The forward pass computes the raw causal score matrix $S \in \mathbb{R}^{T \times T}$:

$$S_{ij} = \begin{cases} \frac{1}{\sqrt{h}} \sum_{d < h} Q_{id} K_{jd} & j \leq i, \\ -\infty & j > i. \end{cases} \quad (92)$$

The attention probability matrix $A \in \mathbb{R}^{T \times T}$ is obtained by applying the row-wise softmax to S :

$$A_{ij} = \text{softmax}(S_{i,\cdot})_j = \frac{e^{S_{ij} - L_i}}{\sum_{l \leq i} e^{S_{il} - L_i}}, \quad (93)$$

where $L_i = \log \sum_{l \leq i} e^{S_{il}}$ is the log-sum-exp (LSE) value for row i . By definition, $A_{ij} = 0$ for $j > i$. The head output $Y \in \mathbb{R}^{T \times h}$ is the matrix product:

$$Y_{id} = \sum_{j < T} A_{ij} V_{jd} \implies Y = AV. \quad (94)$$

Differentiation requires a second, independent set of index names to avoid index collisions. The probe indices are m for position and k for channel.

a) *Local derivative* $\frac{\partial Y}{\partial V}$: Target Y_{id} (query position i , channel d), probe V_{mk} (key position m , channel k):

$$\begin{aligned} \frac{\partial Y_{id}}{\partial V_{mk}} &= \frac{\partial}{\partial V_{mk}} \sum_{a < T} A_{ia} V_{ad} \\ &= \sum_{a < T} A_{ia} \delta_{am} \delta_{dk} \\ &= A_{im} \delta_{dk}. \end{aligned} \quad (95)$$

The column delta sifts the sum down to $a = m$, forcing channel index $d = k$. Chaining the upstream gradient $\frac{\partial L}{\partial Y}$:

$$\begin{aligned} \frac{\partial L}{\partial V_{mk}} &= \sum_{i < T} \sum_{d < h} \frac{\partial L}{\partial Y_{id}} \frac{\partial Y_{id}}{\partial V_{mk}} \\ &= \sum_{i < T} \sum_{d < h} \frac{\partial L}{\partial Y_{id}} A_{im} \delta_{dk} \\ &= \sum_{i < T} A_{im} \frac{\partial L}{\partial Y_{ik}} \\ &= \sum_{i < T} (A^\top)_{mi} \frac{\partial L}{\partial Y_{ik}} = \left(A^\top \frac{\partial L}{\partial Y} \right)_{mk}. \end{aligned} \quad (96)$$

This yields the value gradient matrix product $\frac{\partial L}{\partial V} = A^\top \frac{\partial L}{\partial Y}$.

b) *Local derivative* $\frac{\partial Y}{\partial A}$: Target Y_{id} , probe A_{mj} (query position m , key position j):

$$\begin{aligned} \frac{\partial Y_{id}}{\partial A_{mj}} &= \frac{\partial}{\partial A_{mj}} \sum_{a < T} A_{ia} V_{ad} \\ &= \sum_{a < T} \delta_{im} \delta_{aj} V_{ad} \\ &= \delta_{im} V_{jd}. \end{aligned} \quad (97)$$

The row delta δ_{im} aligns output position i with probed query position m ; the column delta sifts a to j . Chaining the upstream gradient:

$$\begin{aligned} \frac{\partial L}{\partial A_{mj}} &= \sum_{i < T} \sum_{d < h} \frac{\partial L}{\partial Y_{id}} \frac{\partial Y_{id}}{\partial A_{mj}} \\ &= \sum_{i < T} \sum_{d < h} \frac{\partial L}{\partial Y_{id}} \delta_{im} V_{jd} \\ &= \sum_{d < h} \frac{\partial L}{\partial Y_{md}} V_{jd} = \left(\frac{\partial L}{\partial Y} V^\top \right)_{mj}. \end{aligned} \quad (98)$$

This yields the attention probability gradient matrix product $\frac{\partial L}{\partial A} = \frac{\partial L}{\partial Y} V^\top$.

c) *Backpropagation through row-wise Softmax*: Each row of the attention matrix A is an independent softmax over the row of raw scores S . Reusing the softmax result (53) directly for each row i :

$$\frac{\partial L}{\partial S_{ij}} = A_{ij} \left(\frac{\partial L}{\partial A_{ij}} - D_i \right), \quad (99)$$

where the row-wise dot product statistics term is defined as:

$$D_i = \sum_{k < T} \frac{\partial L}{\partial A_{ik}} A_{ik}. \quad (100)$$

Standard backpropagation materializes the full $T \times T$ gradient matrix $\frac{\partial L}{\partial A}$ and computes D_i row-by-row before evaluating $\frac{\partial L}{\partial S_{ij}}$.

d) *Local derivative* $\frac{\partial S}{\partial Q}$: Target score S_{ij} , probe query Q_{mk} (query position m , channel k):

$$\begin{aligned} \frac{\partial S_{ij}}{\partial Q_{mk}} &= \frac{\partial}{\partial Q_{mk}} \left(\frac{1}{\sqrt{h}} \sum_{d < h} Q_{id} K_{jd} \right) \\ &= \frac{1}{\sqrt{h}} \sum_{d < h} \delta_{im} \delta_{dk} K_{jd} \\ &= \frac{1}{\sqrt{h}} \delta_{im} K_{jk}, \end{aligned} \quad (101)$$

where the Kronecker deltas δ_{im} and δ_{dk} enforce query position and channel alignment, sifting the channel sum down to $d = k$. Chaining the score gradient:

$$\begin{aligned} \frac{\partial L}{\partial Q_{mk}} &= \sum_{i < T} \sum_{j < T} \frac{\partial L}{\partial S_{ij}} \frac{\partial S_{ij}}{\partial Q_{mk}} \\ &= \sum_{i < T} \sum_{j < T} \frac{\partial L}{\partial S_{ij}} \frac{1}{\sqrt{h}} \delta_{im} K_{jk} \\ &= \frac{1}{\sqrt{h}} \sum_{j < T} \frac{\partial L}{\partial S_{mj}} K_{jk} \\ &= \left(\frac{1}{\sqrt{h}} \frac{\partial L}{\partial S} K \right)_{mk}, \end{aligned} \quad (102)$$

where the query position delta sifts the outer sum over i down to $i = m$, leaving the contraction over the key position index j , recognized as a standard matrix product $\frac{\partial L}{\partial Q} = \frac{1}{\sqrt{h}} \frac{\partial L}{\partial S} K$.

e) *Local derivative* $\frac{\partial S}{\partial K}$: Target score S_{ij} , probe key K_{mk} (key position m , channel k):

$$\begin{aligned} \frac{\partial S_{ij}}{\partial K_{mk}} &= \frac{\partial}{\partial K_{mk}} \left(\frac{1}{\sqrt{h}} \sum_{d<h} Q_{id} K_{jd} \right) \\ &= \frac{1}{\sqrt{h}} \sum_{d<h} Q_{id} \delta_{jm} \delta_{dk} \\ &= \frac{1}{\sqrt{h}} \delta_{jm} Q_{ik}, \end{aligned} \quad (103)$$

where the row delta δ_{jm} indicates key position alignment, and the channel delta sifts the sum down to $d = k$. Chaining the score gradient:

$$\begin{aligned} \frac{\partial L}{\partial K_{mk}} &= \sum_{i<T} \sum_{j<T} \frac{\partial L}{\partial S_{ij}} \frac{\partial S_{ij}}{\partial K_{mk}} \\ &= \sum_{i<T} \sum_{j<T} \frac{\partial L}{\partial S_{ij}} \frac{1}{\sqrt{h}} \delta_{jm} Q_{ik} \\ &= \frac{1}{\sqrt{h}} \sum_{i<T} \frac{\partial L}{\partial S_{im}} Q_{ik} \\ &= \frac{1}{\sqrt{h}} \sum_{i<T} \left(\frac{\partial L}{\partial S} \right)_{mi} Q_{ik} \\ &= \left(\frac{1}{\sqrt{h}} \frac{\partial L}{\partial S} Q \right)_{mk}, \end{aligned} \quad (104)$$

where the key position delta sifts the inner sum over j down to $j = m$, leaving the contraction over the query position index i , recognized as the matrix product of the transposed score gradient $\frac{\partial L}{\partial S}^\top$ and query matrix Q , yielding $\frac{\partial L}{\partial K} = \frac{1}{\sqrt{h}} \frac{\partial L}{\partial S}^\top Q$. Due to the causal mask, the gradient $\frac{\partial L}{\partial S_{ij}} = 0$ for $j > i$.

f) *FlashAttention-2 Simplification: Softmax Statistics Contraction*: To avoid materializing the intermediate $T \times T$ matrices $\frac{\partial L}{\partial A}$ and $\frac{\partial L}{\partial S}$ in DRAM, FlashAttention-2 [1] simplifies the softmax statistics term D_i . Substituting the definition of $\frac{\partial L}{\partial A_{ik}} = \sum_{d<h} \frac{\partial L}{\partial Y_{id}} V_{kd}$ from (98) into the definition of D_i (100):

$$\begin{aligned} D_i &= \sum_{k<T} \left(\sum_{d<h} \frac{\partial L}{\partial Y_{id}} V_{kd} \right) A_{ik} \\ &= \sum_{d<h} \frac{\partial L}{\partial Y_{id}} \left(\sum_{k<T} A_{ik} V_{kd} \right) \\ &= \sum_{d<h} \frac{\partial L}{\partial Y_{id}} Y_{id}, \end{aligned} \quad (105)$$

where the sum over key positions k in the second line is replaced by the forward pass output Y_{id} using (94). This contraction is a key mathematical optimization: the row statistics term D_i is simply the row-wise dot product of the output Y and its gradient $\frac{\partial L}{\partial Y}$. Precomputing $D_i \in \mathbb{R}^T$ directly from

the output tensors enables evaluating the score gradients $\frac{\partial L}{\partial S_{ij}}$ block-by-block in SRAM:

$$\frac{\partial L}{\partial S_{ij}} = A_{ij} \left(\sum_{d<h} \frac{\partial L}{\partial Y_{id}} V_{jd} - D_i \right), \quad (106)$$

completely eliminating the need to write the large $\frac{\partial L}{\partial A}$ matrix to DRAM.

g) *FlashAttention-2 Memory-Efficient Block Tiling*:

Standard backpropagation requires materializing the $T \times T$ matrices S , A , $\frac{\partial L}{\partial A}$, and $\frac{\partial L}{\partial S}$ in DRAM, creating a massive $O(T^2)$ memory footprint. FlashAttention-2 avoids this bottleneck by tiling the sequence dimension and executing the backward pass in SRAM using two passes:

1) **Pass 1 (dQ Computation)**: Query positions are tiled into blocks of size B_r (indexing rows i). For each block:

- Load Q_i , $\frac{\partial L}{\partial Y_i}$, Y_i , and L_i (the LSE vector) from DRAM to SRAM.
- Compute the row scaling term $D_i = \sum_{d<h} \frac{\partial L}{\partial Y_{id}} Y_{id}$ in SRAM.
- Loop over key/value blocks j of size B_c : load K_j, V_j to SRAM, compute the score block $S_{ij} = Q_i K_j^\top / \sqrt{h}$, mask elements where $j > i$, apply softmax $A_{ij} = \exp(S_{ij} - L_i)$, compute the gradient slice $\frac{\partial L}{\partial S_{ij}} = A_{ij} (\frac{\partial L}{\partial Y_i} V_j^\top - D_i)$, and accumulate:

$$\frac{\partial L}{\partial Q_i} += \frac{1}{\sqrt{h}} \frac{\partial L}{\partial S_{ij}} K_j. \quad (107)$$

- Write the fully accumulated $\frac{\partial L}{\partial Q_i}$ to DRAM once the loop over j completes.

2) **Pass 2 (dK and dV Computation)**: Key and value positions are tiled into blocks of size B_c (indexing columns j). For each block:

- Load K_j, V_j to SRAM and initialize local accumulators $\frac{\partial L}{\partial K_j} = 0, \frac{\partial L}{\partial V_j} = 0$.
- Loop over query blocks i of size B_r : load $Q_i, \frac{\partial L}{\partial Y_i}$, Y_i , and L_i , compute D_i , reconstruct the local slices A_{ij} and $\frac{\partial L}{\partial S_{ij}}$, and accumulate:

$$\frac{\partial L}{\partial V_j} += A_{ij}^\top \frac{\partial L}{\partial Y_i}, \quad (108)$$

$$\frac{\partial L}{\partial K_j} += \frac{1}{\sqrt{h}} \frac{\partial L}{\partial S_{ij}}^\top Q_i. \quad (109)$$

- Write the fully accumulated $\frac{\partial L}{\partial K_j}$ and $\frac{\partial L}{\partial V_j}$ to DRAM once the loop over i completes.

Both passes are completely contention-free (Pass 1 parallelizes over i , Pass 2 over j) and run without any global atomic operations.

Result. For each head, the gradients w.r.t. Queries Q , Keys K , and Values V are:

$$\frac{\partial L}{\partial Q} = \frac{1}{\sqrt{h}} \frac{\partial L}{\partial S} K, \quad (110)$$

$$\frac{\partial L}{\partial K} = \frac{1}{\sqrt{h}} \frac{\partial L}{\partial S}^\top Q, \quad (111)$$

$$\frac{\partial L}{\partial V} = A^\top \frac{\partial L}{\partial Y}, \quad (112)$$

where the causal-masked score gradient $\frac{\partial L}{\partial S} \in \mathbb{R}^{T \times T}$ is:

$$\frac{\partial L}{\partial S_{ij}} = \begin{cases} A_{ij} \left(\frac{\partial L}{\partial A_{ij}} - D_i \right) & j \leq i, \\ 0 & j > i, \end{cases} \quad (113)$$

with the intermediate term and scaling statistics:

$$\frac{\partial L}{\partial A_{ij}} = \sum_{d < h} \frac{\partial L}{\partial Y_{id}} V_{jd}, \quad (114)$$

$$D_i = \sum_{d < h} \frac{\partial L}{\partial Y_{id}} Y_{id}. \quad (115)$$

IX. EMBEDDINGS (ENCODER)

Forward: $x_{bt} = W_E[\text{tok}_{bt}] + W_P[t]$, or in index form:

$$x_{bt,c} = W_{E,\text{tok}_{bt},c} + W_{P,t,c}, \quad (116)$$

where token indices $\text{tok}_{bt} \in \{0, \dots, V-1\}$, token embeddings $W_E \in \mathbb{R}^{V \times C}$, position embeddings $W_P \in \mathbb{R}^{T \times C}$, and channel index $c < C$.

Derivation. Index roles: b batch, t sequence position, c channel. Probing introduces independent indices $k < T$ (position), $v < V$ (vocab token), and $j < C$ (channel).

a) *Gradient w.r.t. position embeddings:* Probing $W_{P,k,j}$ (position k , channel j), the token lookup is constant. The position term differentiates via (5):

$$\frac{\partial W_{P,t,c}}{\partial W_{P,k,j}} = \delta_{tk} \delta_{cj}, \quad (117)$$

leaving the local derivative:

$$\frac{\partial x_{bt,c}}{\partial W_{P,k,j}} = \delta_{tk} \delta_{cj}. \quad (118)$$

Chaining upstream $\frac{\partial L}{\partial x_{bt,c}}$ via (8):

$$\begin{aligned} \frac{\partial L}{\partial W_{P,k,j}} &= \sum_{b < B} \sum_{t < T} \sum_{c < C} \frac{\partial L}{\partial x_{bt,c}} \frac{\partial x_{bt,c}}{\partial W_{P,k,j}} \\ &= \sum_{b < B} \sum_{t < T} \sum_{c < C} \frac{\partial L}{\partial x_{bt,c}} \delta_{tk} \delta_{cj}. \end{aligned} \quad (119)$$

The Kronecker deltas sift and collapse the sums over t and c to $t = k$ and $c = j$. The batch sum survives unconstrained:

$$\frac{\partial L}{\partial W_{P,k,j}} = \sum_{b < B} \frac{\partial L}{\partial x_{bk,j}}, \quad (120)$$

with free indices (k, j) . Sharing $W_{P,t}$ across batches reduces the gradient over the batch dimension b .

b) *Gradient w.r.t. token embeddings:* Probing $W_{E,v,j}$ (vocab token v , channel j) requires comparing the lookup index tok_{bt} against v . In indicator form:

$$W_{E,\text{tok}_{bt},c} = \sum_{u < V} \mathbb{I}[\text{tok}_{bt} = u] W_{E,u,c}. \quad (121)$$

Differentiating with respect to $W_{E,v,j}$:

$$\begin{aligned} \frac{\partial W_{E,\text{tok}_{bt},c}}{\partial W_{E,v,j}} &= \sum_{u < V} \mathbb{I}[\text{tok}_{bt} = u] \frac{\partial W_{E,u,c}}{\partial W_{E,v,j}} \\ &= \sum_{u < V} \mathbb{I}[\text{tok}_{bt} = u] \delta_{uv} \delta_{cj} \\ &= \mathbb{I}[\text{tok}_{bt} = v] \delta_{cj}, \end{aligned} \quad (122)$$

leaving the local derivative:

$$\frac{\partial x_{bt,c}}{\partial W_{E,v,j}} = \mathbb{I}[\text{tok}_{bt} = v] \delta_{cj}. \quad (123)$$

Chaining upstream $\frac{\partial L}{\partial x_{bt,c}}$ via (8):

$$\begin{aligned} \frac{\partial L}{\partial W_{E,v,j}} &= \sum_{b < B} \sum_{t < T} \sum_{c < C} \frac{\partial L}{\partial x_{bt,c}} \frac{\partial x_{bt,c}}{\partial W_{E,v,j}} \\ &= \sum_{b < B} \sum_{t < T} \sum_{c < C} \frac{\partial L}{\partial x_{bt,c}} \mathbb{I}[\text{tok}_{bt} = v] \delta_{cj}. \end{aligned} \quad (124)$$

The delta δ_{cj} sifts the sum over c to j , while the indicator filters the remaining sums over b and t :

$$\frac{\partial L}{\partial W_{E,v,j}} = \sum_{b < B} \sum_{t < T} \frac{\partial L}{\partial x_{bt,j}} \mathbb{I}[\text{tok}_{bt} = v], \quad (125)$$

with free indices (v, j) . This scatter-add accumulates upstream gradients at matching token locations, following the $+=$ convention of §I.

Result. The token and positional embedding gradients are:

$$\frac{\partial L}{\partial W_{E,v,c}} = \sum_{b < B} \sum_{t < T} \frac{\partial L}{\partial x_{bt,c}} \mathbb{I}[\text{tok}_{bt} = v], \quad (126)$$

$$\frac{\partial L}{\partial W_{P,t,c}} = \sum_{b < B} \frac{\partial L}{\partial x_{bt,c}}. \quad (127)$$

$\frac{\partial L}{\partial W_P}$ reduces the upstream gradient over the batch dimension, while $\frac{\partial L}{\partial W_E}$ accumulates contributions via scatter-add at active token locations.

X. THE FULL BACKWARD PASS

Compose §III–§IX in reverse forward order, including the residual stream (where gradients add) and weight tying between W_E and the LM head.

TODO: Write the end-to-end gradient flow for one transformer block, then the whole model.

APPENDIX A GRADIENT CHECKING

Numerical check used to validate every derivation above:

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta + \epsilon e_i) - L(\theta - \epsilon e_i)}{2\epsilon}. \quad (128)$$

TODO: Record per-dtype tolerances and how they relate to `tests/_dtypes.py`.

APPENDIX B USEFUL IDENTITIES

- $\frac{d}{dx} \log x = 1/x$; $\frac{d}{dx} e^x = e^x$; $\frac{d}{dx} \tanh x = 1 - \tanh^2 x$.
- log-sum-exp and its gradient: $\nabla \log \sum_j e^{x_j} = \text{softmax}(x)$.
- **TODO:** Collect identities as they get used.

REFERENCES

- [1] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” *arXiv preprint arXiv:2307.08691*, 2023.